Q Model & Action-States — Implementation Brief

Oct 9, 2026 · @Andy

Purpose and scope

Q is the controller's decision model. Every frame it scores five action-states, and the controller switches to the best valid one. This brief describes the idea and the required behavior; file layout, class design, and code are left to the implementer.

What Q does

- **Reads three input groups,** each through its own branch of a small neural network: an egocentric grid of what is around me, a vector of scalar features, and a confidence map summarizing the belief.
- **Outputs five numbers,** one per action-state: the expected change in score difference over the next 50 turns if the controller enters that state now and then keeps playing as usual.
- **Never overrides safety.** The controller chooses with masks, a switching cost, and a hard safety rule that Q cannot outvote.

Constraints

- The game framework already exists and stays untouched. Agent code lives beside it.
- During a match, only NumPy (and SciPy, if allowed) may run. PyTorch is used offline for training only; the trained network is re-implemented in NumPy for play.
- The belief filter already provides: weighted hypotheses about the enemy, residual (history-unknown) mass, a possibility mask of cells the enemy could occupy, and a position marginal. Q only consumes these outputs.
- Every function that builds an input should be pure, so it can be tested on small hand-built boards.

Terms

Term	Meaning
Score difference	My territory count minus the enemy's territory count
Trail	Cells a player has walked outside its own territory since leaving it
Possibility mask	Every cell the enemy could occupy right now, given the rules and observations
Position marginal	Probability of the enemy being at each cell, from the belief
Residual mass	Belief weight whose history was pruned; its position is known, its trail and territory are not

Per-frame decision loop

Q runs every frame. The order below matters: inputs are built from the updated belief, and the safety rule is applied after Q.

1. Update the belief with the new observation (existing filter).
2. Build the per-frame snapshot: distance maps and safety margins.
3. Build Q's three inputs: the egocentric grid, the scalar features, and the confidence map.
4. Run Q to get five values.
5. Remove action-states that are not valid right now (masks).
6. Switch to the best remaining state only if it beats the current state by the switching cost.
7. Apply the hard rules, which win over steps 5 and 6: after a death, go to PATROL at respawn; if the worst-case safety margin falls to the threshold, go to RETURN.
8. Run the chosen state's own logic for this frame to get one move.
9. Log this frame's inputs, the chosen state, and whether a hard rule forced it.

Action states

Q decides which state to be in; each state's own logic decides how to move. Every state recomputes its target every frame.

State	Purpose	Each frame it…	Valid when
EXPAND	Push outward in a chosen direction	Inside territory: walks to the exit cell chosen by PATROL's exit score. Outside: goes straight outward in the chosen direction. Never turns back; that is RETURN's job.	The next outward cell is on the board
RETURN	Stop expanding and close the loop with the best balance of gain and safety	Replans the way home (below)	Trail length > 0
HUNT	Kill the enemy by stepping on its trail	Targets the enemy trail cell with the best race margin and takes the shortest path there that avoids my own trail	An enemy trail is known (visible, or probability > 0.5)
TRACK TARGET	Keep the enemy in view and gather information	Enemy visible: keeps it 3–5 cells away, inside my view, preferring my own territory. Enemy unseen: heads for the cell whose 11 × 11 view would cover the most belief mass.	Always
PATROL	Stay safe, gather information, find the best place to go out	Walks inside my territory toward the boundary cell with the best exit score, then paces there. Never leaves territory.	Trail length 0

RETURN's replanning. For each sideways length w from 0 to a maximum W, in both sideways directions, form the path "go sideways w cells, then take the shortest path home that avoids my trail." Add the direct shortest path home. For each candidate path

compute:

- **Gain:** cells it would capture by flood fill, with enemy cells counting double.
- **Closing time:** turns until it reaches my territory.
- **Margin:** the worst-case enemy arrival time on any of my exposed cells, minus the closing time.

Take the highest-gain path whose margin is at least the buffer. If none qualifies, take the path with the largest margin. Because this repeats every frame, turning sideways and turning back happen exactly when the best choice changes.

TRACK TARGET when the enemy is unseen. Box-filter the position marginal with an 11 × 11 window. The result at each cell is the belief mass my view would cover from there. Head for the best such cell that is safely reachable, preferring paths through my territory.

PATROL's exit score. For every boundary cell of my territory, combine three things: free cells just outside it (room to expand), the worst-case distance to the enemy (safety), and how long ago the cells beyond it were last seen (information). EXPAND starts from PATROL's best exit cell and heads outward, away from my territory.

State-change rules

Order	Rule
1	After a death, go to PATROL at respawn
2	If the worst-case safety margin is at or below the threshold, go to RETURN
3	Remove invalid states
4	Switch to the best remaining state only if its value exceeds the current state's value by the switching cost h

Rules 1 and 2 override Q. The switching cost stops the controller flipping between states every frame; switching from RETURN back to EXPAND uses a larger cost, because it lengthens the trail. When RETURN closes the loop, trail length becomes 0, RETURN becomes invalid, and the controller moves to the best valid state.

Q inputs: grid and scalars

Egocentric grid: 7 channels, 21 × 21, centered on me. The crop is larger than the 11 × 11 view so my trail fits. Cells off the board are 0 in every channel.

Channel	Contents
0	On the board
1	Visible now (the central 11 × 11)
2	My territory, visible cells
3	My trail, the entire trail as far as the crop reaches
4	Enemy territory, visible cells
5	Enemy trail, visible cells
6	Enemy position, if visible

Scalar features: 22 numbers

#	Feature	Definition	Scaling
1	Trail length	Cells in my trail	÷ 100
2	My count	My territory count	÷ 500
3	Enemy count	Enemy territory count	÷ 500
4	Score difference	My count − enemy count	÷ 500
5	D1: enemy distance to my trail	Taxicab distance from the possibility mask to my nearest trail cell (worst case); 50 if I have no trail	Clip 0–50, ÷ 50
6	D2: my distance to the enemy trail	Shortest path, avoiding my trail, to the nearest known enemy trail cell; 50 if none	Clip 0–50, ÷ 50
7	D3: enemy distance to its territory	Exact if the enemy is visible, otherwise averaged over the position marginal	Clip 0–50, ÷ 50

#	Feature	Definition	Scaling
8	D4: my distance to my territory	Shortest path avoiding my trail	Clip 0–50, ÷ 50
9	D5: my gain if I return now	Flood fill with my shortest path home; enemy cells count double; 0 if no trail	÷ 100
10	D6: enemy gain if it returns now	Same, using the visible enemy trail or the top-weighted hypothesis; my cells count double; 0 if no trail	÷ 100
11	D7: safety margin	D1 − D4	Clip −25 to 25, ÷ 25
12	D8: kill race	D2 − D3	Clip −25 to 25, ÷ 25
13	Turns left	Turns remaining for me	÷ 2000
14	Enemy visible	1 if visible, else 0	—
15	Enemy trail known	1 if any enemy trail cell is known, else 0	—
16	Turns since sighting	Turns since the enemy was last seen, capped at 100	÷ 100
17	Belief spread	log(1 + cells in the possibility mask)	÷ log 2500
18–22	Current state	One-hot over the five states	—

D7 positive means I get home before the enemy can reach my trail. D8 negative means I can reach the enemy's trail before it gets home.

Confidence map

The confidence map summarizes the whole belief as one full-board world state, plus how sure we are about each part of it. It is independent of the other inputs and has its own network branch.

Items per cell. My trail and my position are always certain, so they are left out.

Item	Uncertain because
Enemy territory	Out-of-view captures, deaths
Enemy trail	Unseen movement
My territory	Enemy captures out of view
Enemy position	Unseen movement

Probability of each item at each cell is the weighted vote of all hypotheses, plus a share for residual mass. R is the total residual mass.

Item	From hypotheses	Residual share
Enemy territory	Sum of hypothesis weights whose enemy territory contains the cell	R × 0.5 inside the reach region, otherwise R × the memory map's value
Enemy trail	Sum of hypothesis weights whose enemy trail contains the cell	R × 0.5 inside the reach region, otherwise 0
My territory	Memory map's value, minus the weight of hypotheses whose enemy captured the cell from me	R × 0.5 inside the reach region where the memory map says mine, otherwise R × the memory map's value
Enemy position	The position marginal	Already included

- **The reach region** is a diamond of radius equal to turns since the last sighting, around that sighting, minus the current view. Residual mass has no known history, so it counts as a coin flip anywhere the enemy could have walked.
- **Visible cells** override everything with the observed 0 or 1.
- **For each item:** value = 1 if probability ⩾ 0.5, else 0; confidence = |2 × probability − 1|. Confidence 0 is a coin flip; 1 is certain.
- **Enemy position uses the probability itself.** When the belief is spread out, every cell's value would be 0 with high confidence, which hides where the enemy probably is.

The enemy-trail sum touches every hypothesis and every cell, so compute it in chunks of hypotheses to keep memory small. The territory sums are cheap: one term per distinct territory, of which there are usually few.

Channels and pooling. Build 8 channels on the full 50 × 50 board, then pool 5 × 5 blocks down to 10 × 10 before the network sees them.

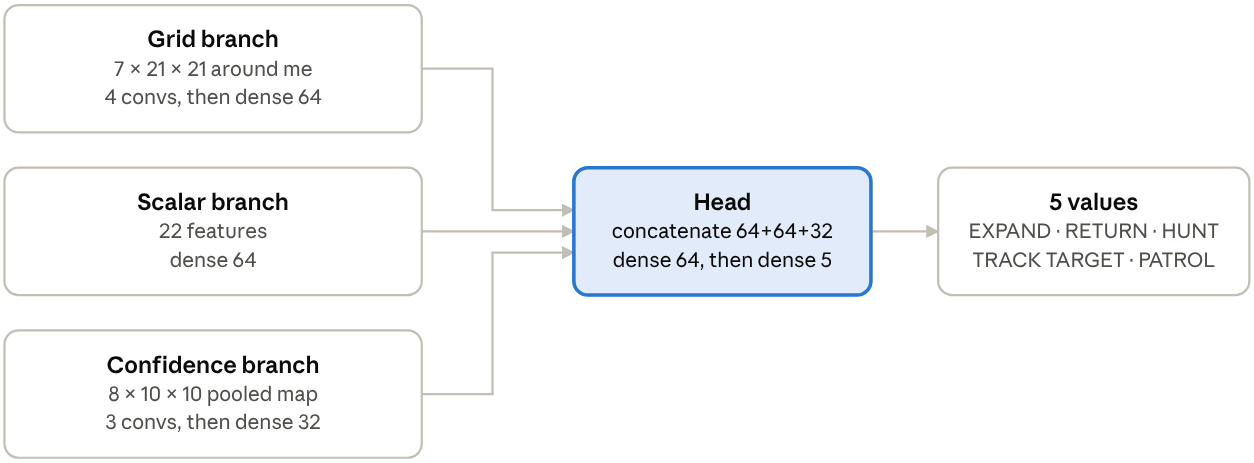
Channel	Contents	Pooling
0, 1, 2	Value: enemy territory, enemy trail, my territory	Mean (fraction of the block believed occupied)
3, 4, 5	Confidence for those three items	Mean (how sure we are about the block)
6	Enemy position probability	Sum (probability the enemy is in the block)
7	My position, one-hot	Sum

Value and confidence stay in separate channels so that, after pooling, a block that is half enemy and half free (but certain) still looks different from a block we know nothing about. My position is included so the branch can relate uncertainty to where I am.

Network structure and output

Three branches, each reading one input group, merge into a shared head with five outputs. The network has about 80,000 parameters.

Three independent branches merge into one head that scores all five states



Q network · three branches, one head, five outputs

Each branch only ever sees its own input, so any one of them can be removed to measure what it adds.

Branch	Input	Layers	Output
Grid	7 × 21 × 21	conv 3×3, 16 → conv 3×3, 32, stride 2 → conv 3×3, 32, stride 2 → conv 3×3, 32, stride 2 → flatten → dense 64	64
Scalars	22	dense 64	64
Confidence	8 × 10 × 10, already pooled	conv 3×3, 16 → conv 3×3, 32, stride 2 → conv 3×3, 32, stride 2 → flatten → dense 32	32
Head	160, concatenated	dense 64 → dense 5	5

All convolutions use padding 1, and every layer except the last uses ReLU. Spatial sizes run 21 → 21 → 11 → 6 → 3 in the grid branch and 10 → 10 → 5 → 3 in the confidence branch.

Output. Five values in the order EXPAND, RETURN, HUNT, TRACK TARGET, PATROL. Each is in cells of score difference over the next 50 turns. Invalid states are masked out before the controller chooses.

Labels and training

Every frame is one training example: the inputs exactly as seen during play, the state actually taken, and what happened to the score difference over the next 50 turns.

Label for the state taken at frame t

```
y_t = \big(\mathrm{score\_diff}_{t+50} - \mathrm{score\_diff}_t\big) +
D5_{t+50}\cdot[\,D7_{t+50} > 0\,]
```

- The second term credits a loop still open when the window ends, but only if it was safe then. Without it, starting a long loop near the end of the window would look worthless.
- Near the end of the game, the window stops at the final turn.
- The state taken is the state the controller is in after that frame's decision. Continuing in the same state counts as choosing it.
- Deaths and kills show up on their own, through the territory counts.

Loss. Squared error between the taken state's output and the label. The other four outputs get no gradient for that example, because their outcomes were never observed. Weight each game equally. Do not use a Huber loss: deaths are large, rare errors, and they are the signal.

Setup. Train offline in PyTorch with Adam at learning rate 1e-3, batch size 256, and weight decay 1e-4. Split train, validation, and test sets by game, never by frame. Stop when validation loss stops improving.

Augmentation. Rotate or flip the whole board: apply the same transform to the grid and to the confidence map. The scalars are counts and distances, so they do not change.

Exploration while collecting games. On about 3% of frames, pick a random valid state and hold it for up to 8 frames, or until it becomes invalid. The hard rules still apply. Without exploration the model never learns what the states it did not choose are worth.

Logging. Store every input exactly as it was during play: the grid as bytes, the scalars as float32, the pooled confidence map as float16 (about 1.6 KB per frame). Never rebuild the confidence map from the true board afterward; that map would be fully confident everywhere, which is not what the model sees in a real game.

Build it in stages

1. Train the scalar branch alone.
2. Add the grid branch; keep it only if validation loss clearly drops.

3. Add the confidence branch under the same test.
4. Collect new games with the trained model, retrain, and repeat 2 to 3 times.

Later improvement. Once a first model exists, replace the end of the window with the previous round's predicted value at t + 50. That gives TRACK TARGET and PATROL credit for information that pays off after more than 50 turns.

Deployment and constants

During a match the controller uses NumPy only.

- Export the trained weights to a NumPy file.
- Re-implement the forward pass in NumPy: convolutions built from sliding windows and matrix multiplies, then the dense layers.
- Check that the NumPy forward pass matches PyTorch within 1e-5 on a set of saved inputs.
- Q runs once per frame with about 80,000 parameters, so its cost is negligible next to the belief filter.

Keep every constant in one configuration file. The values below are starting points to tune.

Constant	Starting value	Used by
Label window N	50 turns	Labels
Switching cost h	2 cells	State changes
Switching cost, RETURN to EXPAND	4 cells	State changes
Safety threshold	1 turn of worst-case margin	Forced RETURN
RETURN buffer	2 turns of margin	RETURN's path choice
RETURN maximum sideways length W	8 cells	RETURN's path choice
TRACK TARGET distance	3–5 cells	TRACK TARGET
Known enemy trail threshold	Probability > 0.5	HUNT, D2

Constant	Starting value	Used by
Grid crop	21 × 21	Grid input
Confidence pooling	5 × 5 blocks, giving 10 × 10	Confidence map
Turns-since-sighting cap	100	Scalar 16
Exploration	3% of frames, held up to 8 frames	Data collection

Acceptance checks and open questions

Inputs

- D5 and D6 are 0 when there is no trail.
- D7 equals D1 − D4, and D8 equals D2 − D3.
- Rotating the board leaves every scalar unchanged.
- Visible cells have confidence 1 for every item.
- A belief with one hypothesis and no residual gives confidence 1 everywhere.
- A belief that is all residual gives enemy-trail confidence 0 throughout the reach region.
- After pooling, the enemy-position and my-position channels each sum to 1.

States

- RETURN is never valid with trail length 0, and PATROL never leaves my territory.
- With the enemy close, RETURN picks the direct path home; with the enemy far away, it picks a wider sideways leg for more gain.
- When the five values are within h of each other, the state never changes.
- A masked state is never chosen.

Model

- The NumPy and PyTorch forward passes agree within 1e-5.
- Per-state calibration: bin predicted values against actual labels, separately for each state.
- Choice frequency: a state chosen never, or almost always, needs its features or masks revisited.
- Win rate against EnemyStateMachine: 200 games per variant in an A/B comparison.

Open questions

- Turn order: does exactly one enemy move happen between my frames? The answer shifts every margin by one turn.
- Are NumPy and SciPy allowed in the submitted controller? Check the starter's candidate guide, or ask robomstr@uw.edu.
- Is 50 turns long enough for most excursions to finish?